FINAL ASSIGNMENT

1. Hello World

First basic program.

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

---

2. Input/Output

Using cin and cout.

```
#include <iostream>
using namespace std;

int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;
    cout << "You entered: " << num << endl;
    return 0;
}
```

---

3. Swap Two Numbers (with temp variable)

Swapping values.

```
#include <iostream>
using namespace std;

int main() {
    int a = 5, b = 10, temp;
    temp = a;
    a = b;
    b = temp;
    cout << "a: " << a << ", b: " << b << endl;
    return 0;
}
```

---

4. Swap Two Numbers (without temp variable)

Swapping values.

```
#include <iostream>
using namespace std;

int main() {
    int a = 5, b = 10;
    a = a + b;
    b = a - b;
    a = a - b;
    cout << "a: " << a << ", b: " << b << endl;
    return 0;
}
```

---

5. Prime Number

Check for prime.

```
#include <iostream>
using namespace std;

int main() {
    int n = 7, i;
    bool isPrime = true;
    if (n <= 1) isPrime = false;
    for (i = 2; i <= n / 2; ++i) {
        if (n % i == 0) {
            isPrime = false;
            break;
        }
    }
    cout << (isPrime ? "Prime" : "Not Prime") << endl;
    return 0;
}
```

---

6. Factorial (using loop)

Using loop or recursion.

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    unsigned long long fact = 1;
    for(int i = 1; i <= n; ++i) {
        fact *= i;
    }
    cout << "Factorial: " << fact << endl;
    return 0;
}
```

---

7. Even or Odd

Check if a number is even or odd.

```
#include <iostream>
using namespace std;

int main() {
    int num = 4;
    if (num % 2 == 0)
        cout << "Even" << endl;
    else
        cout << "Odd" << endl;
    return 0;
}
```

---

8. Leap Year Checker

Check if year is leap.

```
#include <iostream>
using namespace std;

int main() {
    int year = 2024;
    if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
        cout << "Leap Year" << endl;
    else
        cout << "Not Leap Year" << endl;
    return 0;
}
```

---

9. Calculator

Basic arithmetic operations.

```
#include <iostream>
using namespace std;

int main() {
    char op = '+';
    int a = 10, b = 5;
    switch(op) {
        case '+': cout << a + b; break;
        case '-': cout << a - b; break;
        case '*': cout << a * b; break;
        case '/': cout << a / b; break;
    }
    return 0;
}
```

---

10. Reverse a number

Reverse using loop.

```
#include <iostream>
using namespace std;

int main() {
    int n = 1234, rev = 0;
    while(n != 0) {
        rev = rev * 10 + n % 10;
        n /= 10;
    }
    cout << "Reversed: " << rev << endl;
    return 0;
}
```

---

11. Palindrome (Numeric)

Check numeric palindrome.

```
#include <iostream>
using namespace std;

int main() {
    int n = 121, rev = 0, temp = n;
    while (temp != 0) {
        rev = rev * 10 + temp % 10;
        temp /= 10;
    }
    if (n == rev) cout << "Palindrome" << endl;
    else cout << "Not Palindrome" << endl;
    return 0;
}
```

---

12. Palindrome (String)

Check string palindrome.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str = "radar";
    bool pal = true;
    int len = str.length();
    for(int i = 0; i < len/2; i++) {
        if(str[i] != str[len - i - 1]) { pal = false; break; }
    }
    cout << (pal ? "Palindrome" : "Not Palindrome") << endl;
    return 0;
}
```

---

13. Calculator (Menu driven)

Basic arithmetic operations.

```
#include <iostream>
using namespace std;

int main() {
    int choice = 1, a = 10, b = 20;
    if(choice == 1) cout << "Sum: " << a + b << endl;
    return 0;
}
```

---

14. Reverse a number (Loop)

Reverse using loop.

```
#include <iostream>
using namespace std;

int main() {
    int n = 9876, rev = 0;
    while(n > 0) {
        rev = rev * 10 + n % 10;
        n /= 10;
    }
    cout << "Reverse: " << rev << endl;
    return 0;
}
```

---

15. Palindrome (General)

Check numeric/string palindrome.

```
#include <iostream>
using namespace std;

int main() {
    int n = 454, rev = 0, t = n;
    while(t > 0) {
        rev = rev * 10 + t % 10;
        t /= 10;
    }
    if(n == rev) cout << "Palindrome" << endl;
    return 0;
}
```

---

16. Factorial (Recursion)

Recursion.

```
#include <iostream>
using namespace std;

int fact(int n) {
    if(n <= 1) return 1;
    return n * fact(n - 1);
}

int main() {
    cout << "Factorial: " << fact(5) << endl;
    return 0;
}
```

---

17. Fibonacci

Recursive and loop.

```
#include <iostream>
using namespace std;

int main() {
    int n = 5, t1 = 0, t2 = 1, next;
    for (int i = 1; i <= n; ++i) {
        cout << t1 << " ";
        next = t1 + t2;
        t1 = t2;
        t2 = next;
    }
    return 0;
}
```

---

18. GCD/LCM

Using Euclidean algorithm.

```
#include <iostream>
using namespace std;

int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}

int main() {
    int a = 12, b = 18;
    cout << "GCD: " << gcd(a, b) << endl;
    cout << "LCM: " << (a * b) / gcd(a, b) << endl;
    return 0;
}
```

---

19. Class and Object

Create and use simple class.

```
#include <iostream>
using namespace std;

class Demo {
public:
    void greet() { cout << "Hello from Class!" << endl; }
};

int main() {
    Demo obj;
    obj.greet();
    return 0;
}
```

---

20. Constructor & Destructor

Use default, parameterized, and copy constructors.

```
#include <iostream>
using namespace std;

class Box {
public:
    Box() { cout << "Constructor Called" << endl; }
    ~Box() { cout << "Destructor Called" << endl; }
};

int main() {
    Box b;
    return 0;
}
```

---

21. Access Modifiers

Use of private, public, protected.

```
#include <iostream>
using namespace std;

class Test {
private:
    int pri = 10;
public:
    int pub = 20;
};

int main() {
    Test t;
    cout << t.pub << endl;
    return 0;
}
```

---

22. Encapsulation Example

Bundle data and functions.

```
#include <iostream>
using namespace std;

class Encapsulated {
private:
    int x;
public:
    void setX(int val) { x = val; }
    int getX() { return x; }
};

int main() {
    Encapsulated obj;
    obj.setX(42);
    cout << obj.getX() << endl;
    return 0;
}
```

---

23. Data Abstraction

Expose essential features only.

```
#include <iostream>
using namespace std;

class Abstraction {
public:
    void run() {
        init();
        cout << "Running..." << endl;
    }
private:
    void init() { cout << "Internal setup done." << endl; }
};

int main() {
    Abstraction obj;
    obj.run();
    return 0;
}
```

---

24. Inline Functions in Class

Declare and define inline functions inside a class.

```
#include <iostream>
using namespace std;

class Math {
public:
    inline int square(int x) { return x * x; }
};

int main() {
    Math m;
    cout << m.square(5) << endl;
    return 0;
}
```

---

25. Static Data Members & Functions

Use shared variables across objects.

```
#include <iostream>
using namespace std;

class Stat {
public:
    static int count;
    static void show() { cout << "Count: " << count << endl; }
};
int Stat::count = 5;

int main() {
    Stat::show();
    return 0;
}
```

---

26. Friend Function

Access private members using friend.

```
#include <iostream>
using namespace std;

class Base {
private:
    int val = 100;
    friend void printVal(Base b);
};

void printVal(Base b) {
    cout << "Friend Access: " << b.val << endl;
}

int main() {
    Base obj;
    printVal(obj);
    return 0;
}
```

---

27. This Pointer

Use this to resolve name conflicts.

```
#include <iostream>
using namespace std;

class Item {
    int id;
public:
    void setID(int id) { this->id = id; }
    void show() { cout << "ID: " << id << endl; }
};

int main() {
    Item i;
    i.setID(50);
    i.show();
    return 0;
}
```

---

28. Const Member Functions

Declare member functions that don't modify data.

```
#include <iostream>
using namespace std;

class Demo {
    int val = 10;
public:
    int getVal() const { return val; }
};

int main() {
    Demo d;
    cout << d.getVal() << endl;
    return 0;
}
```

---

29. Single Inheritance

One class inherits another.

```
#include <iostream>
using namespace std;

class Parent {
public:
    void load() { cout << "Parent loaded." << endl; }
};

class Child : public Parent {};

int main() {
    Child c;
    c.load();
    return 0;
}
```

---

30. Multiple Inheritance

A class inherits from two classes.

```
#include <iostream>
using namespace std;

class A { public: void msgA() { cout << "A" << endl; } };
class B { public: void msgB() { cout << "B" << endl; } };
class C : public A, public B {};

int main() {
    C obj;
    obj.msgA();
    obj.msgB();
    return 0;
}
```

---

31. Multilevel Inheritance

Inheritance across 3+ levels.

```
#include <iostream>
using namespace std;

class X { public: void show() { cout << "X" << endl; } };
class Y : public X {};
class Z : public Y {};

int main() {
    Z obj;
    obj.show();
    return 0;
}
```

---

32. Hierarchical Inheritance

One base, many derived classes.

```
#include <iostream>
using namespace std;

class Base { public: void info() { cout << "Base" << endl; } };
class Derived1 : public Base {};
class Derived2 : public Base {};

int main() {
    Derived1 d1;
    Derived2 d2;
    d1.info();
    d2.info();
    return 0;
}
```

---

33. Hybrid Inheritance

Combination of multiple types.

```
#include <iostream>
using namespace std;

class World { public: void print() { cout << "World" << endl; } };
class Cont1 : virtual public World {};
class Cont2 : virtual public World {};
class Root : public Cont1, public Cont2 {};

int main() {
    Root r;
    r.print();
    return 0;
}
```

---

34. Function Overloading

Same function name with different arguments.

```
#include <iostream>
using namespace std;

void display(int i) { cout << "Int: " << i << endl; }
void display(double f) { cout << "Double: " << f << endl; }

int main() {
    display(5);
    display(3.14);
    return 0;
}
```

---

35. Operator Overloading

+, -, etc. overloaded for user-defined objects.

```
#include <iostream>
using namespace std;

class Num {
public:
    int val;
    Num(int v) : val(v) {}
    Num operator+(Num const& obj) {
        return Num(val + obj.val);
    }
};

int main() {
    Num n1(10), n2(20);
    Num n3 = n1 + n2;
    cout << "Result: " << n3.val << endl;
    return 0;
}
```

---

36. Function Overriding

Base and derived class same function.

```
#include <iostream>
using namespace std;

class Base {
public:
    void show() { cout << "Base show" << endl; }
};

class Derived : public Base {
public:
    void show() { cout << "Derived show" << endl; }
};

int main() {
    Derived d;
    d.show();
    return 0;
}
```

---

37. Virtual Functions

Use runtime polymorphism.

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual void print() { cout << "Base print" << endl; }
};

class Derived : public Base {
public:
    void print() override { cout << "Derived print" << endl; }
};

int main() {
    Base* b;
    Derived d;
    b = &d;
    b->print();
    return 0;
}
```

---

38. Class Templates

Generic class using templates.

```
#include <iostream>
using namespace std;

template <typename T>
class Hold {
    T val;
public:
    Hold(T v) : val(v) {}
    T get() { return val; }
};

int main() {
    Hold<int> h(123);
    cout << h.get() << endl;
    return 0;
}
```

---

39. Template with Multiple Types

Use multiple parameters.

```
#include <iostream>
using namespace std;

template <class T, class U>
class Pair {
    T first; U second;
public:
    Pair(T f, U s) : first(f), second(s) {}
    void print() { cout << first << " and " << second << endl; }
};

int main() {
    Pair<string, int> p("Age", 21);
    p.print();
    return 0;
}
```

---

40. Write and Read from File

Basic file I/O using fstream.

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream out("test.txt");
    out << "Hello File";
    out.close();

    string s;
    ifstream in("test.txt");
    getline(in, s);
    cout << "Read: " << s << endl;
    return 0;
}
```

---

41. Count Lines/Words in File

Process file content.

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream file("test.txt");
    string word;
    int count = 0;
    while(file >> word) {
        count++;
    }
    cout << "Word count: " << count << endl;
    return 0;
}
```

---

42. Copy One File to Another

Read and write from files.

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream src("test.txt");
    ofstream dst("copy.txt");
    dst << src.rdbuf();
    cout << "File Copied" << endl;
    return 0;
}
```

---

43. Try-Catch Block

Handle division by zero.

```
#include <iostream>
using namespace std;

int main() {
    int b = 0;
    try {
        if(b == 0) throw "Error: Div by 0";
    } catch(const char* msg) {
        cout << msg << endl;
    }
    return 0;
}
```

---

44. Custom Exception

Use throw, try, catch for user-defined errors.

```
#include <iostream>
#include <exception>
using namespace std;

class MyEx : public exception {
public:
    const char* what() const throw() { return "Custom Problem!"; }
};

int main() {
    try {
        throw MyEx();
    } catch(MyEx& e) {
        cout << e.what() << endl;
    }
    return 0;
}
```

---

45. Nested Exception

Multiple catch blocks.

```
#include <iostream>
using namespace std;

int main() {
    try {
        throw 404;
    } catch(const char* e) {
        cout << "String: " << e << endl;
    } catch(int e) {
        cout << "Int error code: " << e << endl;
    }
    return 0;
}
```

---

46. Vector Operations

Insert, delete, sort.

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {4, 1, 3};
    v.push_back(2);
    sort(v.begin(), v.end());
    for(int x : v) cout << x << " ";
    return 0;
}
```

---

47. Map (Key-Value)

Use dictionary-like structure.

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<string, int> m;
    m["Apple"] = 10;
    cout << "Apple count: " << m["Apple"] << endl;
    return 0;
}
```

---

48. Stack/Queue from STL

Use stack, queue, priority_queue.

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> s;
    s.push(10);
    s.push(20);
    cout << "Top: " << s.top() << endl;
    return 0;
}
```

---

49. Set/Multiset

Store unique elements.

```
#include <iostream>
#include <set>
using namespace std;

int main() {
    set<int> s;
    s.insert(10);
    s.insert(10); // duplicate
    cout << "Size: " << s.size() << endl;
    return 0;
}
```

---

50. Constant Data Member Program

Define a constant data member initialized using constructor initializer list.

```
#include <iostream>
using namespace std;

class ConstMember {
public:
    const int val;
    ConstMember(int v) : val(v) {}
};

int main() {
    ConstMember c(100);
    cout << "Const value: " << c.val << endl;
    return 0;
}
```
